一、前置准备

环境要求

- API 接入方式仅需网络环境，无特殊硬件配置要求
- 如需本地部署开源模型，建议配置 GPU 显存 ≥8GB（量化版）或 24GB（标准版）
- 推荐 Python 3.10+（使用 3.10.12 版本可避免兼容性问题）

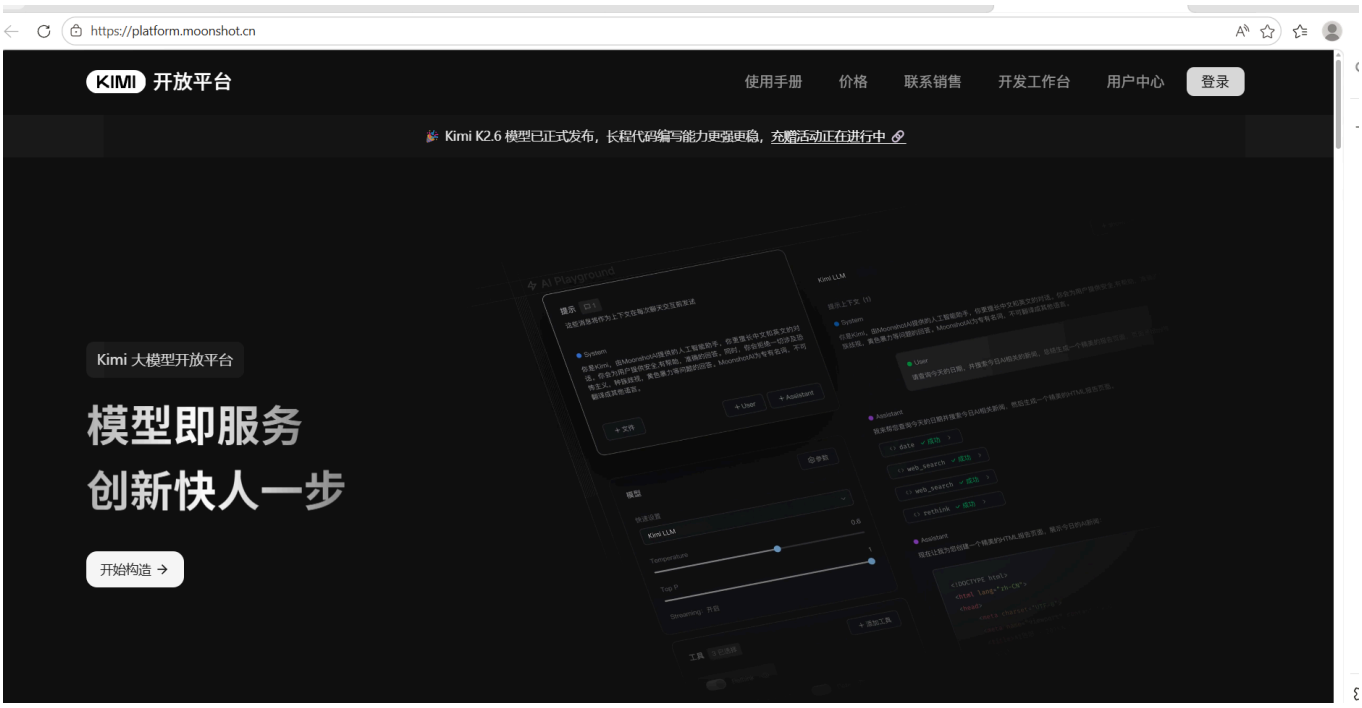
浏览器中食用

访问 Kimi 开放平台 platform.kimi.ai 注册开发者账号，可使用企业邮箱完成注册。新注册用户通常会获赠初始的免费 Token 额度，可先体验再决定是否付费。

二、获取 API Key

步骤 1：注册登录

访问 https://platform.moonshot.cn 或 https://platform.kimi.ai，右上角登录，无账号可用手机号登陆。



步骤 2：进入 API 密钥管理

登录后进入用户中心，找到「API 密钥管理」页面（通常位于“API Keys”选项卡下）。

步骤 3：创建新密钥

点击「创建新密钥」或「Create New Key」按钮，填写应用名称（如 `my-first-api`）后确认生成。请立即复制并保存生成的 API Key（格式为 `sk-xxx`），**该密钥仅显示一次**，关闭后将无法找回。

步骤 4：设置环境变量（推荐）

将 API Key 配置为环境变量，便于后续在代码中安全调用：

bash

export MOONSHOT_API_KEY="sk-your-api-key-here"

在 Windows 系统中可使用 `set MOONSHOT_API_KEY=sk-your-api-key-here` 进行设置。

云端使用建议

云端 API 调用是门槛最低的接入方案，无需关心底层硬件配置。Kimi API 提供 OpenAI 兼容接口，可直接使用标准 OpenAI SDK 调用。如需更高的稳定性与并发能力，亦可考虑通过阿里云百炼平台进行接入。

三、API 服务部署与测试

3.1 HTTP 接口测试（curl）

确认 API Key 有效后，可通过 curl 快速发起一次对话请求：

bash

```
curl -X POST https://api.moonshot.cn/v1/chat/completions
-H "Content-Type: application/json"
-H "Authorization: Bearer $MOONSHOT_API_KEY"
-d '{
"model": "moonshot-v1-8k",
"messages": [
{"role": "user", "content": "你好，请介绍一下自己"}
],
"temperature": 0.7
}'
```

如果一切正常，你将在终端中收到 Kimi 返回的 JSON 格式回复。

3.2 Python 调用（OpenAI SDK）

Kimi API 完全兼容 OpenAI 的接口规范，因此可以直接使用官方 OpenAI Python 库进行调用：

python

```
import os
from openai import OpenAI
client = OpenAI(
api_key=os.environ.get("MOONSHOT_API_KEY"),
base_url="https://api.moonshot.cn/v1",
)
completion = client.chat.completions.create(
model="moonshot-v1-8k",
```

```
messages=[{"role": "user", "content": "请介绍一下 Kimi 的核心能力"}],
temperature=0.7,
)
print(completion.choices[0].message.content)
```

3.3 流式输出（Streaming）

对于需要实时展示生成内容的场景，建议开启流式输出：

python

```
completion = client.chat.completions.create(
model="moonshot-v1-8k",
messages=[{"role": "user", "content": "写一篇关于人工智能发展历程的短文"}],
stream=True,
)
for chunk in completion:
if chunk.choices[0].delta.content:
print(chunk.choices[0].delta.content, end="", flush=True)
```

3.4 Node.js / TypeScript 调用

月之暗面提供了官方 Node.js SDK—— `moonshot-node` ，支持 TypeScript 类型提示，覆盖月之暗面所有开放 API。

安装：

bash

```
npm install moonshot-node
```

使用示例（流式输出）：

javascript

```
const { createSDK } = require("moonshot-node");
const Moonshot = createSDK({
accessToken: process.env.MOONSHOT_API_KEY,
});
const main = async () => {
const result = await Moonshot.chat.createStreamChatCompletion.request({
model: "moonshot-v1-128k",
messages: [{ role: "user", content: "介绍一下月之暗面的技术核心优势" }],
stream: true,
});
```

```
while (true) {
const { done, value } = await result.readContent();
process.stdout.write(value);
if (done) return;
}
};
main();
```

以上代码将逐块输出 Kimi 的回复内容，适用于聊天机器人、实时问答等场景。

3.5 开启 Thinking 深度推理模式（K2.5 模型）

Kimi K2.5 支持特殊的 Thinking 思考模式，**需在请求中显式传入 `enable_thinking: true` 参数才能激活深度推理能力：**

python

```
completion = client.chat.completions.create(
model="kimi-k2.5",
messages=[{"role": "user", "content": "请一步步推理：三个水桶容量分别为8升、5升、3升，如何量出6升水？"}],
extra_body={"enable_thinking": True},
temperature=1.0, # Thinking 模式建议设为 1.0
max_tokens=16000,
)
```

获取思考过程和最终回答

```
for chunk in completion:
if hasattr(chunk.choices[0].delta, "reasoning_content"):
print("[思考]", chunk.choices[0].delta.reasoning_content)
if chunk.choices[0].delta.content:
print("[回答]", chunk.choices[0].delta.content)
```

K2.5 的 Thinking 模式在 AIME 2025 数学基准测试中得分高达 96.1%，是目前开源模型中推理能力最强的功能之一。

3.6 可用模型列表

模型名称	上下文长度	主要特点
`moonshot-v1-8k`	8K	短文本通用对话
`moonshot-v1-32k`	32K	长文本生成与文档处理

模型名称	上下文长度	主要特点
`moonshot-v1-128k`	128K	超长文本处理
`kimi-k2.5 / kimi-k2.6`	256K	多模态、Agent 集群协作、Thinking 深度推理

四、集成到应用框架中

4.1 在 OpenClaw 中配置 Kimi API

OpenClaw 是一款流行的智能体（Agent）框架。将 Kimi 模型接入 OpenClaw，可以显著降低处理本地长文件任务的成本。

找到 OpenClaw 的配置文件（通常位于 `~/.openclaw/config.yaml`），进行如下配置：

yaml

llm:
provider: "openai"
base_url: "https://api.moonshot.cn/v1"
api_key: "你的_Kimi_API_Key"
model: "moonshot-v1-8k"

配置完成后重启 OpenClaw 服务，即可使用 Kimi 模型处理 Agent 任务。如果 Kimi 在调用工具链时输出 JSON 格式不稳定，可在相应 Skill 的系统提示词中强化严格的输出格式约束。

4.2 在 LobeChat / 其他客户端中集成

在 LobeChat 或 LobeHub 等第三方客户端中配置 Kimi API 同样简单：

- 进入客户端的「模型设置」或「第三方 API」页面
- 填入 API Key（`sk-xxx`）和 API 地址（`https://api.moonshot.cn/v1`）
- 选择模型名称（如 `moonshot-v1-128k`）
- 保存配置后即可开始对话

4.3 通过 LangChain 调用

Kimi API 同样可通过 LangChain 框架进行调用：

python

import os
from langchain_community.llms.moonshot import Moonshot

确保环境变量 MOONSHOT_API_KEY 已设置

```
moonshot = Moonshot(model="moonshot-v1-8k")
response = moonshot.invoke("用一段话介绍什么是混合专家模型（MoE）")
print(response)
```

LangChain 库会自动使用 `MOONSHOT_API_KEY` 环境变量进行认证。

五、本地部署方案（开源免费选项）

对于有本地化需求或希望长期规避 API 计费的开发者，可以通过开源项目 `kimi-free-api` 将 Kimi 的服务在本地部署为一个 OpenAI 兼容的网关。该方案支持高速流式输出、多轮对话、联网搜索和文档解读，**与 ChatGPT 接口保持完全兼容**。

5.1 Docker 部署（推荐）

最简单的本地部署方式是使用 Docker：

bash

```
docker run -d --name kimi-free-api -p 8000:8000 vinlic/kimi-free-api:latest
```

服务启动后，本地 API 端点即位于 `http://localhost:8000/v1`，可直接使用 OpenAI SDK 进行调用，只需将 `base_url` 改为本地地址。

5.2 直接启动

bash

```
git clone https://github.com/vinlic/kimi-free-api.git
cd kimi-free-api
npm install
npm start
```

本地化部署的核心优势

价值维度	优势说明
成本控制	完全消除 API 调用费用，降低 90% 以上使用成本
数据安全	对话数据本地存储，满足金融、医疗等行业合规要求
部署灵活	支持 Docker、原生部署等多种方式
功能完整	支持长文本处理、图像识别、联网搜索等全部核心功能

六、定价参考（2026 年 4 月）

官方定价

- 输入：$0.95 / 百万 token
- 输出：$4.0 / 百万 token

Kimi K2.5 / K2.6 采用 MoE 稀疏激活架构——1 万亿总参数，但每个 token 推理时仅激活其中 32B 参数参与计算。这种设计使模型在具备千亿级模型知识广度的同时，保持了 32B 规模的推理速度和成本。

七、常见问题排查

7.1 API Key 无效

- 检查 API Key 格式是否为 `sk-xxx`
- 确认已将 API Key 正确填入 Authorization 请求头（格式：`Bearer sk-xxx`）
- 检查是否在开放平台完成了实名认证

7.2 请求超时或网络不通

- 国内服务器应能正常访问 `https://api.moonshot.cn`，无需特殊网络配置
- 建议将超时时间设置为 60 秒以上

7.3 限流与并发限制

免费接口通常设有严格的速率限制。若收到 HTTP 429 错误，可在客户端降低并发请求数，并配置阶梯式延迟重试策略。